##BFS && DFS:

```
#include <stdio.h>

#include <stdbool.h>

#define MAX_VERTICES 100

// Graph structure

typedef struct {

   int vertices;

   int adjacencyMatrix[MAX_VERTICES][MAX_VERTICES];

} Graph;

// Stack structure for DFS

typedef struct {

   int data[MAX_VERTICES];

   int top;

} Stack;

// Queue structure for BFS

typedef struct {

   int data[MAX_VERTICES];

   int front;

   int rear;

} Queue;

// Initialize graph

void initGraph(Graph* graph, int vertices) {

   graph->vertices = vertices;

   int i, j;

   for (i = 0; i < vertices; i++) {

      for (j = 0; j < vertices; j++) {

         graph->adjacencyMatrix[i][j] = 0;

      }}}

// Add an edge to the graph
```

```
void addEdge(Graph* graph, int source, int destination) {
    graph->adjacencyMatrix[source][destination] = 1;
    graph->adjacencyMatrix[destination][source] = 1;
}
// Depth-first search
void DFS(Graph* graph, int startVertex, bool visited[]) {
    Stack stack;
    stack.top = -1;
    stack.data[++stack.top] = startVertex;
    visited[startVertex] = true;
    printf("DFS: ");
    while (stack.top != -1) {
        int currentVertex = stack.data[stack.top--];
        printf("%d ", currentVertex);
        int i;
        for (i = 0; i < graph->vertices; i++) {
            if (graph->adjacencyMatrix[currentVertex][i] == 1 && !visited[i]) {
                stack.data[++stack.top] = i;
                visited[i] = true;
            } } }
    printf("\n");
}
// Breadth-first search
void BFS(Graph* graph, int startVertex, bool visited[]) {
    Queue queue;
    queue.front = 0;
    queue.rear = -1;
    queue.data[++queue.rear] = startVertex;
    visited[startVertex] = true;
```

```
    printf("BFS: ");
    while (queue.front <= queue.rear) {
        int currentVertex = queue.data[queue.front++];
        printf("%d ", currentVertex);
        int i;
        for (i = 0; i < graph->vertices; i++) {
            if (graph->adjacencyMatrix[currentVertex][i] == 1 && !visited[i]) {
                queue.data[++queue.rear] = i;
                visited[i] = true;
            } }    }
    printf("\n");
}
int main() {
    Graph graph;
    int vertices, edges, i;
    printf("Enter the number of vertices in the graph: ");
    scanf("%d", &vertices);
    initGraph(&graph, vertices);
    printf("Enter the number of edges in the graph: ");
    scanf("%d", &edges);
    for (i = 0; i < edges; i++) {
        int source, destination;
        printf("Enter edge %d (source destination): ", i + 1);
        scanf("%d %d", &source, &destination);
        addEdge(&graph, source, destination);
    }
    bool visited[MAX_VERTICES];
    for (i = 0; i < graph.vertices; i++) {
        visited[i] = false;
```

```
}
int startVertex;
printf("Enter the starting vertex for DFS and BFS: ");
scanf("%d", &startVertex);
DFS(&graph, startVertex, visited);
for (i = 0; i < graph.vertices; i++) {
    visited[i] = false;     }
BFS(&graph, startVertex, visited);
return 0; }
```

##Articulation point

Code:
```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e5+10;
vector<int> adj[N];
bool vis[N];
bool mark[N];
int tin[N];
int low[N];
int timer = 1;

void dfs(int node, int parent)
{
   vis[node] = true;
   tin[node] = low[node] = timer;
   timer++;
   int child = 0;
   for (auto it : adj[node])
```

```
    {
        if (it == parent) continue;
        if (!vis[it])
        {
            dfs(it, node);
            low[node] = min(low[node], low[it]);
            if (low[it] >= tin[node] && parent != -1)
            {
                mark[node] = true;
            }
            child++;
        }
        else
        {
            low[node] = min(low[node], tin[it]);
        }
    }
    if (child > 1 && parent == -1)
    {
        mark[node] = true;
    }
}
vector<int> articulationPoints(int n)
{
    for (int i = 0; i < n; i++)
    {
        if (!vis[i])
        {
            dfs(i, -1);
```

```
        }
    }
    vector<int> ans;
    for (int i = 0; i < n; i++)
    {
        if (mark[i])
        {
            ans.push_back(i);
        }
    }
    if (ans.size() == 0) return { -1};
    return ans;
}
int main()
{
    int n;
    cout<<"Enter the number of edges: ";
    cin >> n;
    cout<<"Enter adjacent list: "<<endl;
    for (int i = 0; i<n; i++)
    {
        int u,v;
        cin>>u>>v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<int> nodes = articulationPoints(n);
    for (auto node : nodes)
    {cout<<"Articulation point = ";
```

```
        cout << node << " ";
    }
    cout << endl;
    return 0;
}
```

######## 0/1 knapshake problem

```
#include<stdio.h>
#include<conio.h>
#include<iostream>

using namespace std;

int max(int a, int b)
{
    return (a > b) ? a : b;
}

int knapSack(int W, int wt[], int val[], int n)
{

    if (n == 0 || W == 0)
        return 0;

    if (wt[n - 1] > W)
        return knapSack(W, wt, val, n - 1);
```

```
    else
        return max(val[n - 1] + knapSack(W - wt[n - 1], wt, val, n - 1),
                knapSack(W, wt, val, n - 1));
}



int main()
{
    cout << "Enter the number of items in a Knapsack:";
    int n, W;
    cin >> n;
    int val[n], wt[n];
    for (int i = 0; i < n; i++)
    {
        cout << "Enter value and weight for item " << i << ":";
        cin >> val[i];
        cin >> wt[i];
    }



    cout << "Enter the capacity of knapsack";
    cin >> W;
    cout << knapSack(W, wt, val, n);


    return 0;
}
```

strongly connected components:

```
#include <bits/stdc++.h>

using namespace std;

vector<int> arr1[51];

vector<int> arr2[51];

int visited[51];

int visited2[51];

stack<int> st;

int r;

void DFS(int node){


   visited[node]=1;

   for(int child: arr1[node]){


      if(visited[child]!=1){

         DFS(child);


      }

   }

   st.push(node);

}

void DFS2(int node){

   cout<<node<<" ";
```

```
    visited2[node]=1;
    for(int child: arr2[node]){
    if(visited2[child]!=1){
            DFS2(child);


        }
    }


}
int main(){


    int n,m,x,y,a;
    cout<<"Enter number of vertex and number of edge"<<endl;
    cin>>n>>m;
    while(m--){
        cout<<"enter vertex having edge in ----> this way"<<endl;
        cin>>x>>y;
        arr1[x].push_back(y);
        arr2[y].push_back(x);
    }
    cout<<"enter starting node"<<endl;
        cin>>a;
    DFS(a);
    cout<<" strongly connected component "<<endl;
    while(!st.empty()){
        int k=st.top();
        st.pop();
        if(visited2[k]!=1){
            r++;
```

```
        cout<<endl;
        DFS2(k);
      }
    }
    cout<<endl;
    cout<<"Total number of strongly connected component = "<<r<<" follows above "<<endl;
    return 0;
}
```

###Dijksta source shortest path

```
#include<bits/stdc++.h>
using namespace std;
int main(){
int n,m,source;
cout << "Enter Node & edges:: ";
cin >> n >> m;
vector<pair<int,int> > g[n+1];
int a,b,wt;
cout << "Enter source & des & wt:: \n";
for(int i = 0; i<m ; i++){
cin >> a >> b >> wt;
g[a].push_back(make_pair(b,wt));
g[b].push_back(make_pair(a,wt));
}
cout << "Enter source:: ";
cin >> source;
```

```
priority_queue<pair<int,int>,vector<pair<int,int> >,greater<pair<int,int> > > pq;// min-heap ; In pair => (dist,from)

vector<int> distTo(n+1,INT_MAX);

distTo[source] = 0;

pq.push(make_pair(0,source));

while( !pq.empty() ){

int dist = pq.top().first;

int prev = pq.top().second;

pq.pop();

vector<pair<int,int> >::iterator it;

for( it = g[prev].begin() ; it != g[prev].end() ; it++){

int next = it->first;

int nextDist = it->second;

if( distTo[next] > distTo[prev] + nextDist){

distTo[next] = distTo[prev] + nextDist;

pq.push(make_pair(distTo[next], next));

}

}

}

cout << "The distances from source, " << source << ", are : \n";

for(int i = 1 ; i<=n ; i++) cout << distTo[i] << " ";

cout << "\n";

return 0;

}
```

##LCS (longest common sub)

```
#include<bits/stdc++.h>

using namespace std;

int LongestCommonSubsequence( string X, string Y, int m, int n )

{

   if (m == 0 || n == 0)

      return 0;

   if (X[m-1] == Y[n-1])

      return 1 + LongestCommonSubsequence(X, Y, m-1, n-1);

   else

      return max(LongestCommonSubsequence(X, Y, m, n-1), LongestCommonSubsequence(X, Y, m-1, n));

}

int main() {

  string s1;

  cin>>s1;

  string s2;

  cin>>s2;

  int a=s1.length();

  int b=s2.length();

  cout<<"Longest common subsequence is:"<<" ";

  cout<<LongestCommonSubsequence(s1,s2,a,b);

  return 0;

}
```